

# **Matrix Computation**

**MATH 3322**

*CHEN, Ruiping*

*Jun 1, 2025*

*rchenbt@connect.ust.hk*

# Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Vector and Matrix Norm</b>                      | <b>3</b>  |
| <b>2</b> | <b>LU Decomposition</b>                            | <b>5</b>  |
|          | <i>LU decomposition using Gaussian elimination</i> |           |
|          | <i>Algorithm for LU decomposition</i>              |           |
|          | <i>Partial pivoting</i>                            |           |
| <b>3</b> | <b>Cholesky Decomposition</b>                      | <b>9</b>  |
|          | <i>Symmetric positive (semi-)definite matrices</i> |           |
|          | <i>Algorithm for Cholesky decomposition</i>        |           |
| <b>4</b> | <b>QR Decomposition</b>                            | <b>12</b> |
|          | <i>Gram-Schmidt algorithm</i>                      |           |
|          | <i>Householder transformation</i>                  |           |
|          | <i>Givens rotation</i>                             |           |
|          | <i>Rank-revealing QR decomposition</i>             |           |
|          | <i>Least square problems</i>                       |           |
|          | <i>Linear regression</i>                           |           |
| <b>5</b> | <b>Eigenvalue Decomposition</b>                    | <b>19</b> |
|          | <i>Eigenvalue and eigenvector</i>                  |           |
|          | <i>Power iteration and its variants</i>            |           |
|          | <i>QR algorithm and tridiagonalization</i>         |           |
|          | <i>Schur decomposition</i>                         |           |
|          | <i>Polynomial root-finding</i>                     |           |
|          | <i>Graph cut</i>                                   |           |
| <b>6</b> | <b>Singular Value Decomposition</b>                | <b>28</b> |
|          | <i>Singular value decomposition</i>                |           |
|          | <i>Algorithms for SVD</i>                          |           |
|          | <i>Least square problems using SVD</i>             |           |
|          | <i>Principal component analysis</i>                |           |
|          | <i>Matrix completion</i>                           |           |

# 1 Vector and Matrix Norm

To measure the magnitude of vectors and matrices, we introduce the concept of **norm**.

*Definition:* A **vector norm** is a function  $\|\cdot\| : \mathbb{R}^n \rightarrow \mathbb{R}$  with the following properties:

1.  $\|\mathbf{x}\| \geq 0 \ \forall \mathbf{x}$ , and  $\|\mathbf{x}\| = 0 \iff \mathbf{x} = \mathbf{0}$
2.  $\|\alpha \mathbf{x}\| = |\alpha| \|\mathbf{x}\|, \forall \alpha \in \mathbb{R} \ \forall \mathbf{x}$
3.  $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\| \ \forall \mathbf{x}, \mathbf{y}$

Among the norms, we're interested in a specific class of norms, called  **$L_p$ -norms**.

*Definition:* We define the  **$L_p$ -norm** as follows

$$\|\mathbf{v}\|_p := \sqrt[p]{\sum_{i=1}^n |v_i|^p}$$

In particular, we define  $\|\mathbf{v}\|_\infty := \max_{1 \leq i \leq n} |v_i|$ , which is derived by taking  $p \rightarrow \infty$ .

The  $L_2$ -norm  $\|\cdot\|_2$  is also called the **Euclidean norm**, which is the "length" of a vector in our natural understanding.

*Remark:* We regard the notation  $\|\cdot\|$  as the  $L_2$ -norm  $\|\cdot\|_2$  by default, if the subscript  $p$  is not explicitly provided.

Moreover, we can extend the definition of vector norms to the matrix space.

*Definition:* A **matrix norm** is a function  $\|\cdot\| : \mathbb{R}^{n \times m} \rightarrow \mathbb{R}$  with the following properties:

1.  $\|\mathbf{A}\| \geq 0 \ \forall \mathbf{A}$ , and  $\|\mathbf{A}\| = 0 \iff \mathbf{A} = \mathbf{O}$
2.  $\|\alpha \mathbf{A}\| = |\alpha| \|\mathbf{A}\|, \forall \alpha \in \mathbb{R} \ \forall \mathbf{A}$
3.  $\|\mathbf{A} + \mathbf{B}\| \leq \|\mathbf{A}\| + \|\mathbf{B}\| \ \forall \mathbf{A}, \mathbf{B}$

The  $L_p$ -norms for matrices are defined as follows:

*Definition:* We define the  **$L_p$ -norm** of matrices as follows

$$\|\mathbf{A}\|_p := \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{Ax}\|_p}{\|\mathbf{x}\|_p}$$

One can view this definition as measuring the maximum magnitude of "stretch" of a vector after being transformed by the matrix.

For some particular  $L_p$ -norms, we have some nice properties:

*Theorem:*

1.  $\|\mathbf{A}\|_1$  = maximum 1-norm of the column vectors of  $\mathbf{A}$
2.  $\|\mathbf{A}\|_\infty$  = maximum 1-norm of the row vectors of  $\mathbf{A}$
3.  $\|\mathbf{A}\|_2$  = maximum singular value of  $\mathbf{A}$

*Theorem:*  $\|\mathbf{AB}\|_p \leq \|\mathbf{A}\|_p \|\mathbf{B}\|_p$

We can also apply the definition of vector  $L_p$ -norms entry-wise on matrices.

*Definition:* We define the "entry-wise" norm of matrices as follows

$$\|\mathbf{A}\|_{\text{vec},p} := \sqrt[p]{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^p}$$

In particular, we denote  $\|\cdot\|_{\text{vec},2}$  by  $\|\cdot\|_F$ , which is called the **Frobenius norm**.

One can regard the "entry-wise" norm as viewing the matrix as a single vector with  $mn$  slots.

## 2 LU Decomposition

Now we introduce the *LU decomposition*.

*Definition:* The *LU decomposition* of a matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$  is a factorization of the form

$$\mathbf{A} = \mathbf{L}\mathbf{U}$$

where  $\mathbf{L} \in \mathbb{R}^{n \times n}$  is lower triangular with diagonals equal to 1,  $\mathbf{U} \in \mathbb{R}^{n \times n}$  is upper triangular.

Before introducing the algorithm of LU decomposition, we first see an application of such decomposition. We take solving linear equations as an example.

*Task:* Given a non-singular matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$  and a vector  $\mathbf{b} \in \mathbb{R}^n$ . We want to solve  $\mathbf{A}\mathbf{x} = \mathbf{b}$ .

One would naturally think of Gaussian elimination. However, if we apply the idea of LU decomposition, the algorithm would be much simpler.

### *Algorithm for solving linear equations using LU decomposition*

1. Perform LU decomposition on  $\mathbf{A}$ . We get  $\mathbf{A} = \mathbf{L}\mathbf{U}$ .
2. Solve  $\mathbf{L}\mathbf{y} = \mathbf{b}$  by back substitution.
3. Solve  $\mathbf{U}\mathbf{x} = \mathbf{y}$  by back substitution.

Another advantage of using LU decomposition is that if one day we need to solve a new equation  $\mathbf{A}\mathbf{x} = \mathbf{b}'$ , we don't need to perform the decomposition step again, which is very efficient. Also, the computation of  $\det \mathbf{A}$  becomes much easier, since

$$\det \mathbf{A} = \det \mathbf{L} \cdot \det \mathbf{U} = \prod_{i=1}^n u_{ii}$$

Now we introduce some algorithms for LU decomposition.

We first take a look at the Gaussian elimination method. The general idea is translating the series of row operations during Gaussian elimination into a sequence of left-multiplications of matrices.

### *Algorithm for LU decomposition using Gaussian elimination*

1. For the  $k$ -th iteration in the Gaussian elimination, we translate the row operation into

a matrix  $\mathbf{L}_k$ , which is in the form  $\mathbf{L}_k = \mathbf{I} - \mathbf{l}_k \mathbf{e}_k^T$ , where  $\mathbf{l}_k = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ * \\ \vdots \\ * \end{bmatrix}$   $\begin{matrix} k \\ \\ n-k \end{matrix}$ .

2. We get  $\mathbf{U} = \mathbf{L}_{n-1} \mathbf{L}_{n-2} \cdots \mathbf{L}_1 \mathbf{A}$ .

3. For each  $\mathbf{L}_k$ , one can show  $\mathbf{L}_k^{-1} = \mathbf{I} + \mathbf{l}_k \mathbf{e}_k^T$ . Compute  $\mathbf{L} = \mathbf{L}_1^{-1} \mathbf{L}_2^{-1} \cdots \mathbf{L}_{n-1}^{-1}$ .

We get  $\mathbf{A} = \mathbf{L}\mathbf{U}$ .

However, this method is generally not used for computing LU decomposition in practice. We introduce a new algorithm, which uses an iterative approach.

For iteration  $k = 1, \dots, n$ , we partition the matrices  $\mathbf{A}$ ,  $\mathbf{L}$  and  $\mathbf{U}$  into 9 partitions along the  $k$ -th row and the  $k$ -th column. For easier references, we use the naming conventions illustrated below:

$$\begin{array}{c}
 \begin{array}{cccc} 1 & \dots & k & \dots \dots n \end{array} \\
 \begin{array}{c} 1 \\ \vdots \\ k \\ \vdots \\ \vdots \\ n \end{array}
 \end{array}
 \left[ \begin{array}{c|c|c} \mathbf{A}_1^{(k)} & \mathbf{a}_2^{(k)} & \mathbf{A}_3^{(k)} \\ \hline \mathbf{a}_4^{(k)} & a_5^{(k)} & \mathbf{a}_6^{(k)} \\ \hline \mathbf{A}_7^{(k)} & \mathbf{a}_8^{(k)} & \mathbf{A}_9^{(k)} \end{array} \right]
 =
 \begin{array}{c}
 \begin{array}{cccc} 1 & \dots & k & \dots \dots n \end{array} \\
 \begin{array}{c} 1 \\ \vdots \\ k \\ \vdots \\ \vdots \\ n \end{array}
 \end{array}
 \left[ \begin{array}{c|c|c} \mathbf{L}_1^{(k)} & & \\ \hline \mathbf{l}_4^{(k)} & l_5^{(k)} & \\ \hline \mathbf{L}_7^{(k)} & \mathbf{l}_8^{(k)} & \mathbf{L}_9^{(k)} \end{array} \right]
 \begin{array}{c}
 \begin{array}{cccc} 1 & \dots & k & \dots \dots n \end{array} \\
 \begin{array}{c} 1 \\ \vdots \\ k \\ \vdots \\ \vdots \\ n \end{array}
 \end{array}
 \left[ \begin{array}{c|c|c} \mathbf{U}_1^{(k)} & \mathbf{u}_2^{(k)} & \mathbf{U}_3^{(k)} \\ \hline & u_5^{(k)} & \mathbf{u}_6^{(k)} \\ \hline & & \mathbf{U}_9^{(k)} \end{array} \right]$$

$\underbrace{\hspace{15em}}_{\mathbf{A}} \qquad \underbrace{\hspace{15em}}_{\mathbf{L}} \qquad \underbrace{\hspace{15em}}_{\mathbf{U}}$

where the subscripts indicate the relative position of the sub-matrices.

We will stick to this naming convention in the following steps.

The algorithm for LU decomposition goes as follows. One can verify its correctness using induction.

***Algorithm for LU decomposition***

$\mathbf{L}, \mathbf{U} \leftarrow \mathbf{O}$

For  $k = 1, 2, \dots, n$

$$l_5^{(k)} \leftarrow 1$$

$$u_5^{(k)} \leftarrow a_5^{(k)} - \mathbf{a}_4^{(k)} \mathbf{a}_2^{(k)}$$

$$\mathbf{l}_8^{(k)} \leftarrow \mathbf{a}_8^{(k)} - \mathbf{A}_7^{(k)} \mathbf{a}_2^{(k)}$$

$$\mathbf{u}_6^{(k)} \leftarrow \mathbf{a}_6^{(k)} - \mathbf{a}_4^{(k)} \mathbf{A}_3^{(k)}$$

$$\mathbf{l}_8^{(k)} \leftarrow \frac{\mathbf{l}_8^{(k)}}{u_5^{(k)}}$$

We get  $\mathbf{A} = \mathbf{LU}$ .

Notice that only the strictly lower-triangular part of  $\mathbf{L}$  and the upper-triangular part of  $\mathbf{U}$  contains useful information. Hence we can modify  $\mathbf{A}$  in place to save memory.

**Algorithm for memory-efficient LU decomposition**

For  $k = 1, 2, \dots, n$

$$a_5^{(k)} \leftarrow a_5^{(k)} - \mathbf{a}_4^{(k)} \mathbf{a}_2^{(k)}$$

$$\mathbf{a}_8^{(k)} \leftarrow \mathbf{a}_8^{(k)} - \mathbf{A}_7^{(k)} \mathbf{a}_2^{(k)}$$

$$\mathbf{a}_6^{(k)} \leftarrow \mathbf{a}_6^{(k)} - \mathbf{a}_4^{(k)} \mathbf{A}_3^{(k)}$$

$$\mathbf{a}_8^{(k)} \leftarrow \frac{\mathbf{a}_8^{(k)}}{a_5^{(k)}}$$

$$\text{Let } (\mathbf{L})_{ij} = \begin{cases} a_{ij} & i > j \\ 1 & i = j \\ 0 & i < j \end{cases}, (\mathbf{U})_{ij} = \begin{cases} a_{ij} & i \leq j \\ 0 & i > j \end{cases}. \text{ We get } \mathbf{A} = \mathbf{LU}.$$

However, when  $a_5^{(k)}$  is small, computing  $\mathbf{a}_8^{(k)} \leftarrow \mathbf{a}_8^{(k)} / a_5^{(k)}$  will cause a large numerical error. Even worse, if we encounter  $a_5^{(k)} = 0$  for some  $k$ , this algorithm cannot even proceed.

To address this, we can use partial pivoting to make the algorithm more numerically stable. The overall idea is to pick the entry with the largest magnitude in  $\begin{bmatrix} a_5^{(k)} \\ \mathbf{a}_8^{(k)} \end{bmatrix}$  and perform a row switch to make the corresponding row "float" up by introducing a permutation matrix  $\mathbf{P} \in \mathbb{R}^{n \times n}$ .

Therefore, we need to slightly change our definition for the **LU decomposition with partial pivoting**.

*Definition:* The **LU decomposition** of a matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$  **with partial pivoting** is a factorization of the form

$$\mathbf{PA} = \mathbf{LU}$$

where  $\mathbf{L} \in \mathbb{R}^{n \times n}$  is lower triangular with diagonals equal to 1,  $\mathbf{U} \in \mathbb{R}^{n \times n}$  is upper triangular, and  $\mathbf{P} \in \mathbb{R}^{n \times n}$  is a permutation matrix.

The algorithm goes as follows:

**Algorithm for memory-efficient LU decomposition with partial pivoting**

$\mathbf{P} \leftarrow \mathbf{I}$

For  $k = 1, 2, \dots, n$

$$a_5^{(k)} \leftarrow a_5^{(k)} - \mathbf{a}_4^{(k)} \mathbf{a}_2^{(k)}$$

$$\mathbf{a}_8^{(k)} \leftarrow \mathbf{a}_8^{(k)} - \mathbf{A}_7^{(k)} \mathbf{a}_2^{(k)}$$

$$\mathbf{A}_{[k][:]} \leftrightarrow \mathbf{A}_{[j][:]}, \mathbf{P}_{[k][:]} \leftrightarrow \mathbf{P}_{[j][:]}, \text{ where } j = \arg_i \max_{k \leq i \leq n} |\mathbf{a}_k^{(i)}|$$

$$\mathbf{a}_6^{(k)} \leftarrow \mathbf{a}_6^{(k)} - \mathbf{a}_4^{(k)} \mathbf{A}_3^{(k)}$$

$$\mathbf{a}_8^{(k)} \leftarrow \frac{\mathbf{a}_8^{(k)}}{a_5^{(k)}}$$

$$\text{Let } (\mathbf{L})_{ij} = \begin{cases} a_{ij} & i > j \\ 1 & i = j \\ 0 & i < j \end{cases}, (\mathbf{U})_{ij} = \begin{cases} a_{ij} & i \leq j \\ 0 & i > j \end{cases}. \text{ We get } \mathbf{PA} = \mathbf{LU}.$$

We can easily verify that in this algorithm, all entries in  $\mathbf{L}$  must have absolute value less or equal to 1.

Also, we can use **full pivoting**  $\mathbf{PAQ} = \mathbf{LU}$  (i.e., choosing pivots from  $\begin{bmatrix} a_5 & \mathbf{a}_6 \\ \mathbf{a}_8 & \mathbf{A}_9 \end{bmatrix}$  instead of only  $\begin{bmatrix} a_5 \\ \mathbf{a}_8 \end{bmatrix}$ ) to make the algorithm even more numerically stable. We'll not discuss this algorithm in detail.



### 3 Cholesky Decomposition

In this section, we'll discuss a new matrix decomposition. Before that, we need to introduce the concept of *symmetric positive definite matrices (SPD)* and *symmetric semi-positive definite matrices (SPSD)*

*Definition:* A matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$  is *symmetric positive definite (SPD)* if

1.  $\mathbf{A} = \mathbf{A}^\top$  (symmetric)
2.  $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$  for all  $\mathbf{x} \neq \mathbf{0}, \mathbf{x} \in \mathbb{R}^n$  (positive definite)

*Definition:* A matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$  is *symmetric semi-positive definite (SPSD)* if

1.  $\mathbf{A} = \mathbf{A}^\top$  (symmetric)
2.  $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$  for all  $\mathbf{x} \neq \mathbf{0}, \mathbf{x} \in \mathbb{R}^n$  (semi-positive definite)

*Theorem:* If a matrix  $\mathbf{A}$  is SPD, then

1.  $\det \mathbf{A} > 0$
2. Principal sub-matrix of  $\mathbf{A}$  is SPD.
3.  $a_{ii} > 0 \quad \forall i$

*Theorem:* If a matrix  $\mathbf{A}$  is SPSP, then

1.  $\det \mathbf{A} \geq 0$
2. Principal sub-matrix of  $\mathbf{A}$  is SPSP.
3.  $a_{ii} \geq 0 \quad \forall i$

*Theorem:* A matrix  $\mathbf{A}$  is SPD if and only if  $\exists \mathbf{C}$  with  $\det \mathbf{C} \neq 0$  such that  $\mathbf{A} = \mathbf{C} \mathbf{C}^\top$ .

*Theorem:* A matrix  $\mathbf{A}$  is SPSP if and only if  $\exists \mathbf{C}$  such that  $\mathbf{A} = \mathbf{C} \mathbf{C}^\top$ .

*Theorem:* A symmetric matrix  $\mathbf{A}$  is SPSP if and only if all eigenvalues of  $\mathbf{A} \geq 0$ .

*Theorem:* A symmetric matrix  $\mathbf{A}$  is SPD if and only if all eigenvalues of  $\mathbf{A} > 0$ .

*Theorem:* The Hessian matrix of a strictly convex function is SPD.

*Theorem:* The Hessian matrix of a convex function is SPSP.

*Theorem:* The covariance matrix of a random vector is SPSP.

*Theorem:*  $\mathbf{A}^\top \mathbf{A}$  is SPSP for any matrix  $\mathbf{A}$ .

Now let's introduce the **Cholesky decomposition**.

*Theorem:* Let  $\mathbf{A} \in \mathbb{R}^{n \times n}$  be SPD. There exists a decomposition  $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ , where  $\mathbf{L} \in \mathbb{R}^{n \times n}$  is a lower triangular matrix with positive diagonals.

*Definition:* The **Cholesky decomposition** of a matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$  is a factorization of the form

$$\mathbf{A} = \mathbf{L}\mathbf{L}^\top$$

where  $\mathbf{L} \in \mathbb{R}^{n \times n}$  is lower triangular with positive diagonals.

Similar to that in LU-decomposition, we partition  $\mathbf{A}$  and  $\mathbf{L}$  as below:

$$\underbrace{\begin{array}{c} 1 \quad \dots \quad k \quad \dots \quad n \\ \begin{array}{c} 1 \\ \vdots \\ k \\ \vdots \\ \vdots \\ n \end{array} \left[ \begin{array}{c|c|c} \mathbf{A}_1^{(k)} & \mathbf{a}_2^{(k)} & \mathbf{A}_3^{(k)} \\ \hline \mathbf{a}_4^{(k)} & a_5^{(k)} & \mathbf{a}_6^{(k)} \\ \hline \mathbf{A}_7^{(k)} & \mathbf{a}_8^{(k)} & \mathbf{A}_9^{(k)} \end{array} \right] \end{array}}_{\mathbf{A}} = \underbrace{\begin{array}{c} 1 \quad \dots \quad k \quad \dots \quad n \\ \begin{array}{c} 1 \\ \vdots \\ k \\ \vdots \\ \vdots \\ n \end{array} \left[ \begin{array}{c|c|c} \mathbf{L}_1^{(k)} & & \\ \hline \mathbf{l}_4^{(k)} & l_5^{(k)} & \\ \hline \mathbf{L}_7^{(k)} & \mathbf{l}_8^{(k)} & \mathbf{L}_9^{(k)} \end{array} \right] \end{array}}_{\mathbf{L}} \underbrace{\begin{array}{c} 1 \quad \dots \quad k \quad \dots \quad n \\ \begin{array}{c} 1 \\ \vdots \\ k \\ \vdots \\ \vdots \\ n \end{array} \left[ \begin{array}{c|c|c} \mathbf{L}_1^{(k)\top} & \mathbf{l}_4^{(k)\top} & \mathbf{L}_7^{(k)\top} \\ \hline & l_5^{(k)} & \mathbf{l}_8^{(k)\top} \\ \hline & & \mathbf{L}_9^{(k)\top} \end{array} \right] \end{array}}_{\mathbf{L}^\top}$$

**Algorithm for Cholesky decomposition**

$\mathbf{L} \leftarrow \mathbf{O}$

For  $k = 1, 2, \dots, n$

$$l_5^{(k)} \leftarrow a_5^{(k)} - \mathbf{a}_4^{(k)} \mathbf{a}_4^{(k)\top}$$

$$\mathbf{l}_8^{(k)} \leftarrow \mathbf{a}_8^{(k)} - \mathbf{A}_7^{(k)} \mathbf{a}_4^{(k)\top}$$

$$l_5^{(k)} \leftarrow \sqrt{l_5^{(k)}}$$

$$\mathbf{l}_8^{(k)} \leftarrow \frac{\mathbf{l}_8^{(k)}}{a_5^{(k)}}$$

We get  $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ .

As in LU-decomposition, we can improve this algorithm by modifying  $\mathbf{A}$  in place.

***Algorithm for memory efficient Cholesky decomposition***

For  $k = 1, 2, \dots, n$

$$a_5^{(k)} \leftarrow a_5^{(k)} - \mathbf{a}_4^{(k)} \mathbf{a}_4^{(k)\top}$$

$$\mathbf{a}_8^{(k)} \leftarrow \mathbf{a}_8^{(k)} - \mathbf{A}_7^{(k)} \mathbf{a}_4^{(k)\top}$$

$$a_5^{(k)} \leftarrow \sqrt{a_5^{(k)}}$$

$$\mathbf{a}_8^{(k)} \leftarrow \frac{\mathbf{a}_8^{(k)}}{a_5^{(k)}}$$

Let  $(\mathbf{L})_{ij} = \begin{cases} a_{ij} & i \geq j \\ 0 & i < j \end{cases}$ . We get  $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ .

## 4 QR Decomposition

We first take a look at the *economic QR decomposition*.

*Definition:* The *economic QR decomposition* of a matrix  $\mathbf{A} \in \mathbb{R}^{m \times n}$  ( $m \geq n$ ) is a factorization of the form

$$\mathbf{A} = \mathbf{Q}\mathbf{R}$$

where the columns of  $\mathbf{Q} \in \mathbb{R}^{m \times n}$  are orthonormal, and  $\mathbf{R} \in \mathbb{R}^{n \times n}$  is upper triangular.

As usual, we partition  $\mathbf{A}$ ,  $\mathbf{Q}$  and  $\mathbf{R}$  as below:

$$\underbrace{\begin{array}{c} 1 \quad \dots \quad k \quad \dots \quad n \\ \left[ \begin{array}{c|c|c} & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \end{array} \right] \\ \mathbf{A}_0^{(k)} \quad \mathbf{a}_k \\ \mathbf{A} \end{array}} = \underbrace{\begin{array}{c} 1 \quad \dots \quad k \quad \dots \quad n \\ \left[ \begin{array}{c|c|c} & & \\ \hline & & \\ \hline & & \\ \hline & & \\ \hline & & \end{array} \right] \\ \mathbf{Q}_0^{(k)} \quad \mathbf{q}_k \\ \mathbf{Q} \end{array}} \underbrace{\begin{array}{c} 1 \quad \dots \quad k \quad \dots \quad n \\ \begin{array}{c} 1 \\ \vdots \\ k \\ \vdots \\ n \end{array} \left[ \begin{array}{c|c|c} & & \\ \hline \mathbf{R}_1^{(k)} & \mathbf{r}_2^{(k)} & \\ \hline & r_5^{(k)} & \\ \hline & & \end{array} \right] \\ \mathbf{R} \end{array}}$$

Now let's introduce the *Gram-Schmidt algorithm*.

*The Gram-Schmidt algorithm*

$\mathbf{Q}, \mathbf{R} \leftarrow \mathbf{O}$

For  $k = 1, \dots, n$

$$\mathbf{r}_2^{(k)} \leftarrow \mathbf{A}_0^{(k)\top} \mathbf{a}_k$$

$$\mathbf{q}_k \leftarrow \mathbf{a}_k - \mathbf{A}_0^{(k)} \mathbf{r}_2^{(k)}$$

$$r_5^{(k)} \leftarrow \|\mathbf{q}_k\|$$

$$\mathbf{q}_k \leftarrow \frac{\mathbf{q}_k}{r_5^{(k)}}$$

We get  $\mathbf{A} = \mathbf{Q}\mathbf{R}$ .

Similarly, we can modify  $\mathbf{A}$  in place to save memory.

### *Memory efficient Gram-Schmidt algorithm*

$$\mathbf{R} \leftarrow \mathbf{O}$$

For  $k = 1, \dots, n$

$$\mathbf{r}_2^{(k)} \leftarrow \mathbf{A}_0^{(k)\top} \mathbf{a}_k$$

$$\mathbf{a}_k \leftarrow \mathbf{a}_k - \mathbf{A}_0^{(k)} \mathbf{r}_2^{(k)}$$

$$r_5^{(k)} \leftarrow \|\mathbf{a}_k\|$$

$$\mathbf{a}_k \leftarrow \frac{\mathbf{a}_k}{r_5^{(k)}}$$

$$\mathbf{Q} \leftarrow \mathbf{A}. \text{ We get } \mathbf{A} = \mathbf{QR}.$$

However, this algorithm may not be numerically stable. The algorithm can be modified by further exploiting the orthogonality of the columns of  $\mathbf{Q}$ , which leads to the **modified Gram-Schmidt Algorithm**.

### *Modified Gram-Schmidt algorithm*

$$\mathbf{R} \leftarrow \mathbf{O}$$

For  $k = 1, \dots, n$

For  $i = 1, \dots, k - 1$

$$r_{ik} \leftarrow \mathbf{a}_i^\top \mathbf{a}_k$$

$$\mathbf{a}_k \leftarrow \mathbf{a}_k - r_{ik} \mathbf{a}_i$$

$$r_5^{(k)} \leftarrow \|\mathbf{a}_k\|$$

$$\mathbf{a}_k \leftarrow \frac{\mathbf{a}_k}{r_5^{(k)}}$$

$$\mathbf{Q} \leftarrow \mathbf{A}. \text{ We get } \mathbf{A} = \mathbf{QR}.$$

Note that in the economic QR decomposition, we let  $\mathbf{Q} \in \mathbb{R}^{m \times n}$  be of the same shape as  $\mathbf{A}$ , and  $\mathbf{R} \in \mathbb{R}^{n \times n}$  be square. If we let  $\mathbf{Q} \in \mathbb{R}^{m \times m}$  be square and  $\mathbf{R} \in \mathbb{R}^{m \times n}$  be of the shape as  $\mathbf{A}$ , we get the **full QR decomposition**.

*Definition:* The **full QR decomposition** of a matrix  $\mathbf{A} \in \mathbb{R}^{m \times n}$  is a factorization of the form

$$\mathbf{A} = \mathbf{QR}$$

where  $\mathbf{Q} \in \mathbb{R}^{m \times m}$  is orthogonal,  $\mathbf{R} \in \mathbb{R}^{m \times n}$  is upper triangular.

The algorithm for full QR decomposition uses **Householder transformation**.

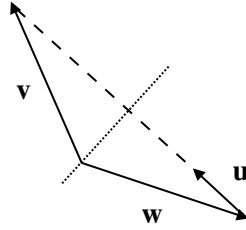
*Definition:* Given  $\mathbf{v}, \mathbf{w} \in \mathbb{R}^m$  with  $\|\mathbf{v}\| = \|\mathbf{w}\|$ , the **Householder matrix**  $\mathbf{H} \in \mathbb{R}^{m \times m}$  is an orthogonal matrix satisfying  $\mathbf{H}\mathbf{v} = \mathbf{w}$ . The transformation from  $\mathbf{v}$  to  $\mathbf{w}$  is called the **Householder transformation**.

*Theorem:* Given  $\mathbf{v}, \mathbf{w} \in \mathbb{R}^m$  with  $\|\mathbf{v}\| = \|\mathbf{w}\|$ , the Householder matrix  $\mathbf{H}$  is given by

$$\mathbf{H} = \mathbf{I} - 2\mathbf{u}\mathbf{u}^\top$$

where  $\mathbf{u} = \frac{\mathbf{v} - \mathbf{w}}{\|\mathbf{v} - \mathbf{w}\|}$ .

One can view the Householder matrix  $\mathbf{H}$  as a "mirror" that reflects  $\mathbf{v}$  to  $\mathbf{w}$ .



Now let's introduce the full QR decomposition using Householder transform.

The general idea is to reduce the entries below the main diagonal of  $\mathbf{A}$  to 0 column by column using Householder transform. By doing so repeatedly, we can eventually get a lower triangular matrix  $\mathbf{R}$ . And since the transformation matrix in every step is orthogonal, by multiplying them up, we get our orthogonal matrix  $\mathbf{Q}$ .

To be more specific, in the  $k$ -th iteration ( $k = 1, \dots, n$ ), we reduce the vector  $\mathbf{A}_{[k+1:m][k]}$  to  $c\mathbf{e}_1$ , while keeping  $\mathbf{A}_{[1:k][k]}$  unchanged. For example, in the second iteration, we want to turn

$$\begin{bmatrix} * & * & * & \dots & * \\ 0 & * & * & \dots & * \\ 0 & * & * & \dots & * \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & * & * & \dots & * \end{bmatrix} \quad \text{to} \quad \begin{bmatrix} * & * & * & \dots & * \\ 0 & * & * & \dots & * \\ 0 & 0 & * & \dots & * \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & * & \dots & * \end{bmatrix}$$

Let  $\mathbf{v} = \mathbf{A}_{[k+1:m][k]}$  and  $\mathbf{w} = c\mathbf{e}_1$ , we can apply the Householder transform. Since in Householder transformation we need  $\|\mathbf{A}_{[k+1:m][k]}\| = \|c\mathbf{e}_1\|$ , hence we should have  $c = \pm\|\mathbf{A}_{[k+1:m][k]}\|$ . Although theoretically the sign of  $c$  does not matter, in practice, however, we usually pick the sign consistent with  $a_{kk}$  for the consideration of numerical stability.

Hence in each step, the transformation matrix  $\mathbf{Q}_k$  should be a concatenation of an identity matrix and a Householder matrix, namely

$$\mathbf{Q}_k = \begin{bmatrix} \mathbf{I}_{k-1} & \\ & \mathbf{H}_{\mathbf{a}_k^{(k)}} \end{bmatrix} \in \mathbb{R}^{n \times n}$$

where

$$\mathbf{H}_{\mathbf{a}_k^{(k)}} = \mathbf{I} - 2\mathbf{u}_k\mathbf{u}_k^\top$$

where

$$\mathbf{u}_k = \frac{\mathbf{a}_{k[k:n]} + \text{sgn}(a_{kk})\|\mathbf{a}_{k[k:n]}\| \cdot \mathbf{e}_1}{\left\| \mathbf{a}_{k[k:n]} + \text{sgn}(a_{kk})\|\mathbf{a}_{k[k:n]}\| \cdot \mathbf{e}_1 \right\|} \in \mathbb{R}^{n-k+1}$$

The matrix  $\mathbf{Q}$  is given by the product of the transformation matrices in each step, namely

$$\mathbf{Q} = (\mathbf{Q}_n \mathbf{Q}_{n-1} \cdots \mathbf{Q}_1)^\top = \mathbf{Q}_1 \mathbf{Q}_2 \cdots \mathbf{Q}_n$$

The full algorithm goes as follows:

**Algorithm for QR decomposition using Householder transformation**

For  $k = 1, 2, \dots, n$

$$\mathbf{u}_k \leftarrow \mathbf{a}_{k[k:n]} + \text{sgn}(a_{kk})\|\mathbf{a}_{k[k:n]}\| \cdot \mathbf{e}_1$$

$$\mathbf{u}_k \leftarrow \frac{\mathbf{u}_k}{\|\mathbf{u}_k\|}$$

$$\mathbf{A}_{[k:m][k:n]} \leftarrow \mathbf{A}_{[k:m][k:n]} - 2\mathbf{u}_k \mathbf{u}_k^\top \mathbf{A}_{[k:m][k:n]}$$

$$\mathbf{Q} \leftarrow \mathbf{I}$$

For  $k = n, n-1, \dots, 1$

$$\mathbf{Q}_{[k:m][1:m]} \leftarrow \mathbf{Q}_{[k:m][1:m]} - 2\mathbf{u}_k \mathbf{u}_k^\top \mathbf{Q}_{[k:m][1:m]}$$

$$\mathbf{R} \leftarrow \mathbf{A}. \text{ We get } \mathbf{A} = \mathbf{QR}.$$

Apart from Householder transformation, we can also use **Givens rotation** for QR decomposition.

The general idea is to eliminating the entries below the main diagonal of  $\mathbf{A}$  to 0 one by one using Givens rotation.

For example, at the  $k$ -th step, we want to turn the matrix

$$\begin{bmatrix} * & * & * & * & * \\ & * & * & * & * \\ & & x_1 & * & * \\ & & x_2 & * & * \\ & & & * & * \end{bmatrix} \quad \text{to} \quad \begin{bmatrix} * & * & * & * & * \\ & * & * & * & * \\ & & * & * & * \\ & & & * & * \\ & & & & * \end{bmatrix}$$

To achieve this, let  $\mathbf{G}(\theta) = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}$ , where  $\cos \theta = \frac{x_1}{\sqrt{x_1^2 + x_2^2}}$  and  $\sin \theta = \frac{x_2}{\sqrt{x_1^2 + x_2^2}}$ .

One can check that

$$\mathbf{G}(\theta) \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} \sqrt{x_1^2 + x_2^2} \\ 0 \end{bmatrix}$$

Hence

$$\underbrace{\begin{bmatrix} \mathbf{I}_2 & & \\ & \mathbf{G}(\theta) & \\ & & 1 \end{bmatrix}}_{:=\mathbf{G}_k} \begin{bmatrix} * & * & * & * & * \\ & * & * & * & * \\ & & x_1 & * & * \\ & & x_2 & * & * \\ & & & * & * \end{bmatrix} = \begin{bmatrix} * & * & * & * & * \\ & * & * & * & * \\ & & * & * & * \\ & & & * & * \\ & & & & * \end{bmatrix}$$

Such transformation is called a **Givens rotation**.

One can check that for each step  $k$ , the transformation matrix  $\mathbf{G}_k = \begin{bmatrix} \mathbf{I} & & \\ & \mathbf{G}(\theta) & \\ & & \mathbf{I} \end{bmatrix}$  is orthogonal.

Assume we performed  $p$  iterations to reduce  $\mathbf{A}$  to upper triangular, namely  $\mathbf{G}_p \mathbf{G}_{p-1} \cdots \mathbf{G}_1 \mathbf{A} = \mathbf{R}$ .

Then let  $\mathbf{Q} = \mathbf{G}_1^\top \mathbf{G}_2^\top \cdots \mathbf{G}_p^\top$ , we get the QR decomposition  $\mathbf{A} = \mathbf{QR}$ .

The Givens rotation method is especially efficient if  $\mathbf{A}$  is sparse (i.e., most entries are zeros).

If the columns of  $\mathbf{A}$  are highly collinear, we can perform the **rank-revealing QR decomposition**.

*Definition:* The **rank-revealing QR decomposition** of a matrix  $\mathbf{A} \in \mathbb{R}^{m \times n}$  is a factorization of the form

$$\mathbf{AP} = \mathbf{QR}$$

or more specifically,

$$\mathbf{AP} = \begin{bmatrix} || & || \\ \mathbf{A}_1 & \mathbf{A}_2 \\ || & || \\ k & n-k \end{bmatrix} = \begin{bmatrix} || & || & * \\ \mathbf{Q}_1 & \mathbf{Q}_2 & * \\ || & || & * \\ k & n-k & m-n \end{bmatrix} \begin{bmatrix} \mathbf{R}_{11} & \mathbf{R}_{12} \\ & \mathbf{R}_{22} \\ k & n-k \end{bmatrix} \begin{matrix} k \\ n-k \\ m-n \end{matrix}$$

where  $\mathbf{P} \in \mathbb{R}^{n \times n}$  is a permutation,  $\mathbf{Q} \in \mathbb{R}^{m \times m}$  is orthogonal,  $\mathbf{R} \in \mathbb{R}^{m \times n}$  is upper triangular satisfying  $\|\mathbf{R}_{22}\|_F < \varepsilon$  is small.  $k$  is called the **numerical rank** of  $\mathbf{A}$ .

One can observe that  $\mathbf{A}_1 = \mathbf{Q}_1 \mathbf{R}_{11}$  is just the economic QR of  $\mathbf{A}_1$ , and  $\mathbf{A}_2 = \mathbf{Q}_1 \mathbf{R}_{12} + \mathbf{Q}_2 \mathbf{R}_{22} \approx \mathbf{Q}_1 \mathbf{R}_{12}$ .

Columns in  $\mathbf{A}_1$  can be seen as the representatives of  $\mathbf{A}$ , in the sense that  $\text{range}(\mathbf{A}_1) \approx \text{range}(\mathbf{A})$ .

This decomposition can be done by many software packages. We'll not discuss the algorithm in detail.

Now let's see some applications of QR decomposition.

*Task:* Given a non-singular matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$  and a vector  $\mathbf{b} \in \mathbb{R}^n$ . We want to solve  $\mathbf{Ax} = \mathbf{b}$ .

Given the QR decomposition  $\mathbf{A} = \mathbf{QR}$ , we can have

$$\mathbf{Ax} = \mathbf{b} \iff \mathbf{QRx} = \mathbf{b} \iff \mathbf{Rx} = \mathbf{Q}^\top \mathbf{b}$$

which is easy to solve by back substitution.



**Algorithm for solving linear equations using QR decomposition**

1. Perform QR decomposition on  $\mathbf{A}$ . We get  $\mathbf{A} = \mathbf{QR}$ .
2. Compute  $\mathbf{y} = \mathbf{Q}^\top \mathbf{b}$ .
3. Solve  $\mathbf{R}\mathbf{x} = \mathbf{y}$  by back substitution.

Now assume  $\mathbf{A} \in \mathbb{R}^{m \times n}$  with  $m > n$ . The equation set  $\mathbf{Ax} = \mathbf{b}$  generally has no solution. Our aim becomes solving  $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|^2$ . Such problems are called *least square problems*.

*Task:* Given a matrix  $\mathbf{A} \in \mathbb{R}^{m \times n}$  with  $m > n$  and a vector  $\mathbf{b} \in \mathbb{R}^n$ . We want to solve

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|^2$$

The solution is usually denoted by  $\hat{\mathbf{x}}$ .

*Theorem:* The solution to least square problems is the solution to

$$\mathbf{A}^\top \mathbf{A} \hat{\mathbf{x}} = \mathbf{A}^\top \mathbf{b} \quad (\text{normal equation})$$

In other word, we have

$$\hat{\mathbf{x}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$$

**Algorithm for solving least square problems using normal equation**

1. Compute  $\mathbf{A}^\top \mathbf{A}$ .
2. Compute  $\mathbf{A}^\top \mathbf{b}$ .
3. Solve  $\mathbf{A}^\top \mathbf{A} \hat{\mathbf{x}} = \mathbf{A}^\top \mathbf{b}$ .

Now consider the QR decomposition  $\mathbf{A} = \mathbf{QR}$ . Since  $\mathbf{QQ}^\top$  is a projector onto  $\text{Range}(\mathbf{A})$ , we have

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|^2 \iff \mathbf{QR}\hat{\mathbf{x}} = \mathbf{QQ}^\top \mathbf{b} \iff \mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^\top \mathbf{b}$$

Therefore, we can solve  $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|^2$  by solving  $\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^\top \mathbf{b}$ .

**Algorithm for solving least square problems using QR decomposition**

1. Perform economic QR decomposition on  $\mathbf{A}$ . We get  $\mathbf{A} = \mathbf{QR}$ .
2. Compute  $\mathbf{y} = \mathbf{Q}^\top \mathbf{b}$ .
3. Solve  $\mathbf{R}\hat{\mathbf{x}} = \mathbf{y}$  by back substitution.

The algorithm using QR method is generally more numerically stable than the one using normal equation. To understand this, we need to introduce the concept of *condition number*.

*Definition:* The **condition number** of  $\mathbf{A}$  is given by

$$\kappa_{\mathbf{A}} = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})}$$

where  $\sigma_{\max}(\mathbf{A})$  and  $\sigma_{\min}(\mathbf{A})$  are the maximum and minimum singular values of  $\mathbf{A}$ , respectively.

The larger  $\kappa_{\mathbf{A}}$  is, the less numerical stability  $\mathbf{A}$  has.

One can view  $\frac{1}{\kappa_{\mathbf{A}}}$  as the "distance" of  $\mathbf{A}$  to the set of singular matrices.

*Theorem:* Given a matrix  $\mathbf{A} \in \mathbb{R}^{m \times n}$  with  $m > n$  and its QR decomposition  $\mathbf{A} = \mathbf{Q}\mathbf{R}$ , we have

$$\kappa_{\mathbf{P}} = \kappa_{\mathbf{R}}^2$$

where  $\mathbf{P} = \mathbf{A}^{\top} \mathbf{A}$ .

In general,  $\kappa_{\mathbf{P}} > \kappa_{\mathbf{R}}$ . Therefore we can conclude that the algorithm using QR method is generally more numerically stable.

We conclude this section by one last example.

*Task:* Given  $m$  pairs of data points  $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ , find a function  $f$  such that  $y_i = f(\mathbf{x}_i)$ . We consider  $f$  that can be expressed by  $n$  ( $n < m$ ) basis functions, i.e.,  $f(\mathbf{x}) = \sum_{j=1}^n c_j \psi_j(\mathbf{x})$ .

This is the generalized version of linear regression.

$$\text{We define } \mathbf{A} = \begin{bmatrix} \psi_1(\mathbf{x}_1) & \psi_2(\mathbf{x}_1) & \cdots & \psi_n(\mathbf{x}_1) \\ \psi_1(\mathbf{x}_2) & \psi_2(\mathbf{x}_2) & \cdots & \psi_n(\mathbf{x}_2) \\ \vdots & \vdots & \ddots & \vdots \\ \psi_1(\mathbf{x}_m) & \psi_2(\mathbf{x}_m) & \cdots & \psi_n(\mathbf{x}_m) \end{bmatrix}, \mathbf{c} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix}, \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

Our goal is to minimize  $\sum_{i=1}^m (y_i - \sum_{j=1}^n c_j \psi_j(\mathbf{x}_i))^2$ . In other word, we want to solve

$$\min_{\mathbf{c} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{c} - \mathbf{y}\|^2$$

which is exactly a least square problem and can be solved using the QR method discussed before.

## 5 Eigenvalue Decomposition

First let's state the definition of the *eigenvalues* and *eigenvectors* of a matrix.

*Definition:* A non-zero vector  $\mathbf{x}$  is an *eigenvector* of  $\mathbf{A}$  with *eigenvalue*  $\lambda \in \mathbb{C}$  if

$$\mathbf{A}\mathbf{x} = \lambda \mathbf{x}$$

Note that the eigenvalues and eigenvectors of a real matrix can be complex.

*Definition:* The set of eigenvalues of  $\mathbf{A}$  is called the *spectrum* of  $\mathbf{A}$ . We define the *spectral radius*

$$\rho(\mathbf{A}) = \max \{ |\lambda| : \lambda \in \text{spectrum of } \mathbf{A} \}$$

*Theorem:* If  $(\lambda, \mathbf{x})$  is an eigenpair of  $\mathbf{A}$ , then

1.  $(\lambda^2, \mathbf{x})$  is an eigenpair of  $\mathbf{A}^2$ .
2.  $(\lambda - \sigma, \mathbf{x})$  is an eigenpair of  $\mathbf{A} - \sigma \mathbf{I}$ .
3.  $(\frac{1}{\lambda - \sigma}, \mathbf{x})$  is an eigenpair of  $(\mathbf{A} - \sigma \mathbf{I})^{-1}$ .
4.  $(p(\lambda), \mathbf{x})$  is an eigenpair of  $p(\mathbf{A})$  for any polynomial  $p(x)$ .

*Definition:* Two matrices  $\mathbf{A}$  and  $\mathbf{B}$  are *similar* with each other if there exists a nonsingular matrix  $\mathbf{T}$  such that

$$\mathbf{B} = \mathbf{T}\mathbf{A}\mathbf{T}^{-1}$$

*Theorem:* Similar matrices have the same set of eigenvalues.

*Definition:* The *eigenvalue decomposition* of a matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$  is a factorization of the form

$$\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$$

where  $\mathbf{\Lambda} \in \mathbb{C}^{n \times n}$  is diagonal,  $\mathbf{X} \in \mathbb{C}^{n \times n}$ .

We say  $\mathbf{A}$  is *diagonalizable* if it can be written in this form. The columns of  $\mathbf{X}$  and the corresponding diagonal entries of  $\mathbf{\Lambda}$  are the eigenvectors and eigenvalues of  $\mathbf{A}$ .

*Definition:* The *characteristic polynomial* of  $\mathbf{A} \in \mathbb{R}^{n \times n}$ , denoted by  $p_{\mathbf{A}}$ , is an  $n$ -degree polynomial defined by

$$p_{\mathbf{A}}(z) = \det(z\mathbf{I} - \mathbf{A}), \quad z \in \mathbb{C}$$

One can easily show the following theorem:

*Theorem:*  $\lambda$  is an eigenvalue of  $\mathbf{A}$  if and only if  $p_{\mathbf{A}}(\lambda) = 0$ .

Therefore, finding the eigenvalues of  $\mathbf{A}$  is equivalent to finding the roots of  $p_{\mathbf{A}}(x) = 0$ .

Moreover, if  $\mathbf{A} \in \mathbb{R}^{n \times n}$  is symmetric, we have some nice properties.

*Theorem:* If  $\mathbf{A}$  is symmetric, then its eigenvalue decomposition has a special form  $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ , where  $\mathbf{Q}$  is orthogonal, and  $\mathbf{\Lambda}$  is a real diagonal matrix.

In this section, We mainly discuss the eigenvalue decomposition of symmetric matrices.

Let  $\mathbf{A} \in \mathbb{R}^{n \times n}$  be symmetric. We sort its eigenvalues in magnitude  $|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_n|$ , with the corresponding eigenvectors forming an orthogonal matrix  $\mathbf{Q} = [\mathbf{x}_1 \ \mathbf{x}_2 \ \dots \ \mathbf{x}_n] \in \mathbb{R}^{n \times n}$ . We will stick to this naming convention in the following parts.

Before introducing the algorithm for eigenvalue decomposition, we introduce the **Rayleigh quotient**, which enables us to estimate the eigenvalue given the corresponding eigenvector.

*Definition:* The **Rayleigh quotient** of  $\mathbf{A} \in \mathbb{R}^{n \times n}$  is defined as

$$r_{\mathbf{A}}(\mathbf{x}) = \frac{\mathbf{x}^\top \mathbf{A} \mathbf{x}}{\mathbf{x}^\top \mathbf{x}}$$

*Theorem:*  $\mathbf{x}_i$  is an eigenvector of  $\mathbf{A}$  if and only if  $r_{\mathbf{A}}(\mathbf{x}_i) = \lambda_i$ .

*Theorem:*

$\max_{\mathbf{x} \neq 0} r_{\mathbf{A}}(\mathbf{x})$  is equal to the maximum eigenvalue of  $\mathbf{A}$ .

$\min_{\mathbf{x} \neq 0} r_{\mathbf{A}}(\mathbf{x})$  is equal to the minimum eigenvalue of  $\mathbf{A}$ .

Let's begin with some easy tasks.

*Task:* Given a symmetric matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$ , find the largest eigenvalue  $\lambda_1$  (in magnitude) and the corresponding eigenvector  $\mathbf{x}_1$  with  $\|\mathbf{x}_1\| = 1$ .

To obtain the largest eigenvalue (in magnitude), we introduce the power iteration.

#### **Algorithm of power iteration**

1. Choose  $\mathbf{y}^{(0)} \in \mathbb{R}^n$  such that  $\|\mathbf{y}^{(0)}\| = 1$ .

2. For  $k = 1, 2, \dots$  until convergence

$$\mathbf{z}^{(k)} \leftarrow \mathbf{A} \mathbf{y}^{(k-1)}$$

$$\mathbf{y}^{(k)} \leftarrow \frac{\mathbf{z}^{(k)}}{\|\mathbf{z}^{(k)}\|}$$

$$\mu^{(k)} \leftarrow \mathbf{y}^{(k)\top} \mathbf{A} \mathbf{y}^{(k)}$$

We get the eigenpair  $(\mu^{(k)}, \mathbf{y}^{(k)})$ .

*Remark:* Power iteration may fail to converge if  $|\lambda_1| = |\lambda_2|$  or  $\mathbf{x}_1^\top \mathbf{y}^{(0)} = 0$ .

Now, say, we want to obtain the eigenvalue  $\lambda_n$  with the smallest magnitude. A good observation is that  $\frac{1}{\lambda_n}$  is the largest eigenvalue of  $\mathbf{A}^{-1}$ . Based on this fact, we can apply the *inverse power iteration*.

*Task:* Given a symmetric matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$ , find the smallest eigenvalue  $\lambda_n$  (in magnitude) and the corresponding eigenvector  $\mathbf{x}_n$  with  $\|\mathbf{x}_n\| = 1$ .

**Algorithm of inverse power iteration**

1. Choose  $\mathbf{y}^{(0)} \in \mathbb{R}^n$  such that  $\|\mathbf{y}^{(0)}\| = 1$ .

2. For  $k = 1, 2, \dots$  until convergence

$$\mathbf{z}^{(k)} \leftarrow \mathbf{A}^{-1} \mathbf{y}^{(k-1)} \quad (\text{in practice we solve } \mathbf{A} \mathbf{z}^{(k)} = \mathbf{y}^{(k-1)} \text{ using LU method})$$

$$\mathbf{y}^{(k)} \leftarrow \frac{\mathbf{z}^{(k)}}{\|\mathbf{z}^{(k)}\|}$$

$$\mu^{(k)} \leftarrow \mathbf{y}^{(k)\top} \mathbf{A} \mathbf{y}^{(k)}$$

We get the eigenpair  $(\mu^{(k)}, \mathbf{y}^{(k)})$ .

*Remark:* Inverse power iteration may fail to converge if  $|\lambda_n| = |\lambda_{n-1}|$  or  $\mathbf{x}_n^\top \mathbf{y}^{(0)} = 0$ .

Now we want to find an arbitrary eigenpair of  $\mathbf{A}$  based on an estimation of  $\lambda_j$ .

*Task:* Given a symmetric matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$  and an estimation  $\mu$ , find the eigenvalue  $\lambda_j$  closest to  $\mu$  and the corresponding eigenvector  $\mathbf{x}_j$  with  $\|\mathbf{x}_j\| = 1$ .

Similar to our observation in the previous task, we can find that  $\frac{1}{\lambda_j - \mu}$  is the largest eigenvalue of  $(\mathbf{A} - \mu \mathbf{I})^{-1}$ .

**Algorithm of shifted inverse power iteration**

1. Choose  $\mathbf{y}^{(0)} \in \mathbb{R}^n$  such that  $\|\mathbf{y}^{(0)}\| = 1$ .

2. For  $k = 1, 2, \dots$  until convergence

$$\mathbf{z}^{(k)} \leftarrow (\mathbf{A} - \mu \mathbf{I})^{-1} \mathbf{y}^{(k-1)}$$

$$\mathbf{y}^{(k)} \leftarrow \frac{\mathbf{z}^{(k)}}{\|\mathbf{z}^{(k)}\|}$$

$$\mu^{(k)} \leftarrow \mathbf{y}^{(k)\top} \mathbf{A} \mathbf{y}^{(k)}$$

We get the eigenpair  $(\mu^{(k)}, \mathbf{y}^{(k)})$ .

*Remark:* Shifted inverse power iteration may fail to converge if  $\exists k \quad |\lambda_j - \mu| = |\lambda_k - \mu|$  or  $\mathbf{x}_j^\top \mathbf{y}^{(0)} = 0$ .

Moreover, we can slightly modify the algorithm by updating our estimation of  $\mu$  in each iteration to accelerate convergence. The modified algorithm is called the **Rayleigh quotient iteration**.

**Algorithm of Rayleigh quotient iteration**

1. Choose  $\mathbf{y}^{(0)} \in \mathbb{R}^n$  such that  $\|\mathbf{y}^{(0)}\| = 1$ . Set  $\mu^{(0)} \leftarrow \mu$ .

2. For  $k = 1, 2, \dots$  until convergence

$$\mathbf{z}^{(k)} \leftarrow (\mathbf{A} - \mu^{(k-1)}\mathbf{I})^{-1} \mathbf{y}^{(k-1)} \quad (\text{use the latest estimation } \mu^{(k-1)})$$

$$\mathbf{y}^{(k)} \leftarrow \frac{\mathbf{z}^{(k)}}{\|\mathbf{z}^{(k)}\|}$$

$$\mu^{(k)} \leftarrow \mathbf{y}^{(k)\top} \mathbf{A} \mathbf{y}^{(k)}$$

We get the eigenpair  $(\mu^{(k)}, \mathbf{y}^{(k)})$ .

*Remark:* Rayleigh quotient iteration may fail to converge if  $\exists k \mid \lambda_j - \mu \mid = \mid \lambda_k - \mu \mid$  or  $\mathbf{x}_j^\top \mathbf{y}^{(0)} = 0$ .

So far, we can only obtain one specific eigenpair of the matrix. What if we want the top  $r$  eigenvalues?

*Task:* Given a symmetric matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$ , find the top  $r \leq n$  eigenvalues and corresponding eigenvectors of  $\mathbf{A}$ .

The idea is similar, just feed in multiple vectors at the same time. We perform QR decomposition in each iteration to force the vectors to be orthonormal.

**Algorithm of simultaneous power iteration**

1. Choose  $\mathbf{Y}^{(0)} \in \mathbb{R}^{n \times r}$  such that  $\mathbf{Y}^{(0)\top} \mathbf{Y}^{(0)} = \mathbf{I}$ .

2. For  $k = 1, 2, \dots$  until convergence

$$\mathbf{Z}^{(k)} \leftarrow \mathbf{A} \mathbf{Y}^{(k-1)}$$

Compute the economic QR decomposition  $\mathbf{Z}^{(k)} = \mathbf{Y}^{(k)} \mathbf{R}^{(k)}$ .

For  $i = 1, 2, \dots, r$

$$\mu_i^{(k)} \leftarrow \mathbf{y}_i^{(k)\top} \mathbf{A} \mathbf{y}_i^{(k)} \quad (\text{where } \mathbf{Y}^{(k)} = [\mathbf{y}_1^{(k)} \quad \mathbf{y}_2^{(k)} \quad \dots \quad \mathbf{y}_r^{(k)}])$$

We get the eigenpairs  $(\mu_i^{(k)}, \mathbf{y}_i^{(k)})_{i=1}^r$ .

Finally, let's introduce the algorithm for eigenvalue decomposition.

*Task:* Given a symmetric matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$ . Factorize  $\mathbf{A}$  of the form

$$\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$$

where  $\mathbf{Q}$  is orthogonal and  $\mathbf{\Lambda}$  is diagonal.

***Algorithm for eigenvalue decomposition***

1. Initialize  $\mathbf{A}^{(0)} \leftarrow \mathbf{A}$

2. For  $k = 1, 2, \dots$  until convergence

Compute QR decomposition  $\mathbf{A}^{(k-1)} = \mathbf{Q}^{(k)}\mathbf{R}^{(k)}$ .

$$\mathbf{A}^{(k)} \leftarrow \mathbf{R}^{(k)}\mathbf{Q}^{(k)}$$

$$\mathbf{\Lambda} \leftarrow \mathbf{A}^{(k)}, \mathbf{Q} \leftarrow \mathbf{Q}^{(1)} \dots \mathbf{Q}^{(k)}.$$

We get the eigenvalue decomposition  $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ .

However, the computational cost of QR decomposition of  $\mathbf{A}^{(k)}$  is high. To avoid this, we want to reduce  $\mathbf{A}^{(0)} = \mathbf{Q}^{(0)\top} \mathbf{A} \mathbf{Q}^{(0)}$  to tridiagonal, so that QR decomposition can be done by Givens rotation with a much smaller computational cost.

We can use Householder transformation for *tridiagonalization*.

***Algorithm for tridiagonalization***

For  $k = 1, \dots, n - 2$

$$\mathbf{v}_k \leftarrow \mathbf{A}_{[k+1:n][k]}$$

$$\mathbf{u}_k \leftarrow \mathbf{v}_k + \text{sgn}(\mathbf{v}_k[1])\|\mathbf{v}_k\| \cdot \mathbf{e}_1$$

$$\mathbf{u}_k \leftarrow \frac{\mathbf{u}_k}{\|\mathbf{u}_k\|}$$

$$\mathbf{A}_{[k+1][k]}, \mathbf{A}_{[k][k+1]} \leftarrow -\text{sgn}(\mathbf{v}_k[1])\|\mathbf{v}_k\|$$

$$\mathbf{w}_k = \mathbf{A}_{[k+1:n][k+1:n]}\mathbf{u}_k$$

$$\mathbf{A}_{[k+1:n][k+1:n]} \leftarrow \mathbf{A}_{[k+1:n][k+1:n]} - 2\mathbf{u}_k\mathbf{w}_k^\top - 2\mathbf{w}_k\mathbf{u}_k^\top + 4\mathbf{u}_k\mathbf{w}_k^\top\mathbf{w}_k\mathbf{u}_k^\top$$

After tridiagonalizing  $\mathbf{A}^{(0)}$ , by induction, we can prove that all  $\mathbf{A}^{(k)}$  are tridiagonal. Thus, the QR decomposition of  $\mathbf{A}^{(k)}$  can be done by Givens rotation with computational complexity  $O(n)$ .

The full algorithm goes as follows:

**Algorithm for eigenvalue decomposition using tridiagonalization**

1. Tridiagonalize  $\mathbf{A}$ . We get  $\mathbf{A}^{(0)} \leftarrow \mathbf{Q}^{(0)\top} \mathbf{A} \mathbf{Q}^{(0)}$ .
2. For  $k = 1, 2, \dots$  until convergence
 

Compute QR decomposition  $\mathbf{A}^{(k-1)} = \mathbf{Q}^{(k)} \mathbf{R}^{(k)}$  (using Givens rotation)

$\mathbf{A}^{(k)} \leftarrow \mathbf{R}^{(k)} \mathbf{Q}^{(k)}$

$\mathbf{\Lambda} \leftarrow \mathbf{A}^{(k)}, \mathbf{Q} \leftarrow \mathbf{Q}^{(1)} \dots \mathbf{Q}^{(k)}$ .

We get the eigenvalue decomposition  $\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top$ .

So far, we have restricted  $\mathbf{A}$  to symmetric matrices. For many non-symmetric matrices, we cannot diagonalize it in the form  $\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top$  in general. Instead, we can perform **Schur decomposition**.

*Definition:* Given a matrix  $\mathbf{A} \in \mathbb{R}^{n \times n}$ , the **Schur decomposition** is a factorization of the form

$$\mathbf{A} = \mathbf{Q} \mathbf{S} \mathbf{Q}^\top$$

where  $\mathbf{Q} \in \mathbb{R}^{n \times n}$  is orthogonal, and  $\mathbf{S} \in \mathbb{R}^{n \times n}$  is a block upper triangular matrix with diagonal blocks being  $1 \times 1$  or  $2 \times 2$ , i.e.,

$$\mathbf{S} = \begin{bmatrix} \mathbf{S}_{11} & \mathbf{S}_{12} & \cdots & \mathbf{S}_{1p} \\ & \mathbf{S}_{22} & \cdots & \mathbf{S}_{2p} \\ & & \ddots & \vdots \\ & & & \mathbf{S}_{pp} \end{bmatrix}$$

where  $\mathbf{S}_{ii} \in \mathbb{R}^{1 \times 1}$  or  $\mathbb{R}^{2 \times 2} \forall i = 1, \dots, p$ . If  $\mathbf{S}_{ii} \in \mathbb{R}^{2 \times 2}$  then  $\mathbf{S}_{ii}$  must be in the form  $\begin{bmatrix} a & -b \\ b & a \end{bmatrix}$ , and  $a \pm bi$  must be eigenvalues of  $\mathbf{A}$ . If  $\mathbf{S}_{ii} \in \mathbb{R}^{1 \times 1}$ , then  $\mathbf{S}_{ii}$  must be an eigenvalue of  $\mathbf{A}$ .

For a non-symmetric matrix  $\mathbf{A}$ , we cannot tridiagonalize  $\mathbf{A}$  as in eigenvalue decomposition of symmetric matrices. Instead, we can transform  $\mathbf{A}$  into an upper-Hessenberg matrix  $\mathbf{A}^{(0)}$ , namely

$$\mathbf{A}^{(0)} = \mathbf{Q}^{(0)\top} \mathbf{A} \mathbf{Q}^{(0)} = \begin{bmatrix} * & \cdots & * & * \\ * & \cdots & * & * \\ & \ddots & \vdots & \vdots \\ & & * & * \end{bmatrix}$$

Applying QR algorithm to  $\mathbf{A}$  will get Schur decomposition  $\mathbf{A} = \mathbf{Q} \mathbf{S} \mathbf{Q}^\top$ .

However, power iteration and its variants (i.e., inverse iteration, shifted inverse iteration, Rayleigh quotient) can still be extended straightforwardly to any non-symmetric matrix.

Let's see some applications of eigenvalue decomposition.

*Task:* Let  $p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1} + x^n$ . We want to find the roots of  $p(x) = 0$ .



We construct

$$\mathbf{A}_p = \begin{bmatrix} 0 & 0 & \cdots & 0 & -a_0 \\ 1 & 0 & \cdots & 0 & -a_1 \\ 0 & 1 & \cdots & 0 & -a_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & -a_{n-1} \end{bmatrix}$$

One can show that  $p(x) = \det(x\mathbf{I} - \mathbf{A}_p)$ .

Hence the eigenvalues of  $\mathbf{A}_p$  are roots of  $p(x) = 0$ . We can find the eigenvalues of  $\mathbf{A}_p$  using the algorithm discussed before.

Our next example is about **graph cut**.

*Task:* Given data points  $\{\mathbf{x}_i\}_{i=1}^m, \mathbf{x}_i \in \mathbb{R}^n$ . We want to separate them into two clusters.

We first construct a weighted graph  $G = (V, E, w)$  where

$$V = \{\mathbf{x}_i\}_{i=1}^m \quad E = \{\{\mathbf{x}_i, \mathbf{x}_j\}\}_{i,j=1}^m \quad w_{ij} = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{\sigma^2}\right)$$

Define  $\text{cut}(A, B) := \sum_{i \in A, j \in B} w_{ij}$ . We want to solve  $\min_{S \subset V} \text{cut}(S, \bar{S})$ , where  $\bar{S} = V \setminus S$ .

Moreover, we define  $\mathbf{z} \in \mathbb{R}^m$  with  $z_i = \begin{cases} 1 & i \in S \\ -1 & i \in \bar{S} \end{cases}$ ,  $\mathbf{W} = [w_{ij}]_{i,j=1}^m$ ,  $\mathbf{D} = \text{diag}(\mathbf{W}\mathbf{1})$ ,  $\mathbf{L} = \mathbf{D} - \mathbf{W}$ .

The matrix  $\mathbf{L}$  is called the **Laplacian matrix** of the graph  $G = (V, E, w)$ . One can verify the following theorems.

*Theorem:*  $\mathbf{L}$  is SPSPD and  $(0, \mathbf{1})$  is an eigenpair of  $\mathbf{L}$ .

*Theorem:*  $\text{cut}(S, \bar{S}) = \frac{1}{2} \mathbf{z}^\top \mathbf{L} \mathbf{z}$

Hence

$$\min_{S \subset V} \text{cut}(S, \bar{S}) \iff \min_{\forall i | z_i| = 1} \mathbf{z}^\top \mathbf{L} \mathbf{z}$$

With proper relaxation of the constraint, we have

$$\min_{S \subset V} \text{cut}(S, \bar{S}) \xrightarrow{\text{relaxation}} \min_{\substack{\mathbf{z}^\top \mathbf{1} = 0 \\ \|\mathbf{z}\| = \sqrt{m}}} \mathbf{z}^\top \mathbf{L} \mathbf{z} \quad (*)$$

One can check that the solution to  $(*)$  is  $\mathbf{z} = \sqrt{m} \mathbf{x}_{m-1}$ , where  $\mathbf{x}_{m-1}$  is the eigenvector of the second smallest eigenvalue of  $\mathbf{L}$  with  $\|\mathbf{x}_{m-1}\| = 1$ .

We set  $S = \{\mathbf{x}_i : z_i > 0\}$ ,  $\bar{S} = \{\mathbf{x}_i : z_i < 0\}$  and get the resulting clusters.

However, such algorithm may give trivial solutions, e.g. the resulting  $S$  may only contain one single data point. To avoid this, we introduce the **normalized cut**.

We define  $\text{Vol}(A) := \sum_{i \in A, j \in V} w_{ij} = \sum_{i \in A} d_{ii}$  and  $\text{Ncut}(A, B) := \frac{\text{cut}(A, B)}{\text{Vol}(A)} + \frac{\text{cut}(A, B)}{\text{Vol}(B)}$ , which penalizes unbalanced clusters. We want to solve  $\min_{S \subset V} \text{Ncut}(S, \bar{S})$ .

Moreover, we define  $\mathbf{z} \in \mathbb{R}^m$  with  $z_i = \begin{cases} \frac{1}{\text{Vol}(S)} & i \in S \\ -\frac{1}{\text{Vol}(\bar{S})} & i \in \bar{S} \end{cases}$   $\mathbf{W} = [w_{ij}]_{i,j=1}^m$ ,  $\mathbf{D} = \text{diag}(\mathbf{W}\mathbf{1})$ .

Define  $\tilde{\mathbf{L}} = \mathbf{D}^{-\frac{1}{2}} \mathbf{L} \mathbf{D}^{-\frac{1}{2}} = \mathbf{I} - \mathbf{D}^{-\frac{1}{2}} \mathbf{W} \mathbf{D}^{-\frac{1}{2}}$ . The matrix  $\tilde{\mathbf{L}}$  is called the *normalized Laplacian matrix*.

One can verify the following theorems.

*Theorem:*  $\tilde{\mathbf{L}}$  is SPSD and  $(0, \mathbf{D}^{\frac{1}{2}}\mathbf{1})$  is an eigenpair of  $\tilde{\mathbf{L}}$ .

*Theorem:*  $\text{Ncut}(S, \bar{S}) = \frac{\mathbf{z}^\top \mathbf{L} \mathbf{z}}{\mathbf{z}^\top \mathbf{D} \mathbf{z}}$

Hence

$$\min_{S \subset V} \text{Ncut}(S, \bar{S}) \iff \min_{z_i \in \left\{ \frac{1}{\text{Vol}(S)}, -\frac{1}{\text{Vol}(\bar{S})} \right\}} \frac{\mathbf{z}^\top \mathbf{L} \mathbf{z}}{\mathbf{z}^\top \mathbf{D} \mathbf{z}}$$

With proper relaxation of the constraint, we have

$$\min_{S \subset V} \text{Ncut}(S, \bar{S}) \xRightarrow{\text{relaxation}} \min_{\mathbf{z}^\top \mathbf{D} \mathbf{1} = 0} \frac{\mathbf{z}^\top \mathbf{L} \mathbf{z}}{\mathbf{z}^\top \mathbf{D} \mathbf{z}} \quad (\star)$$

Let  $\mathbf{y} = \mathbf{D}^{\frac{1}{2}} \mathbf{z}$ . We have

$$\min_{\mathbf{z}^\top \mathbf{D} \mathbf{1} = 0} \frac{\mathbf{z}^\top \mathbf{L} \mathbf{z}}{\mathbf{z}^\top \mathbf{D} \mathbf{z}} \iff \min_{\mathbf{y}^\top \mathbf{D}^{1/2} \mathbf{1} = 0} \frac{\mathbf{y}^\top \tilde{\mathbf{L}} \mathbf{y}}{\mathbf{y}^\top \mathbf{y}}$$

One can check that the solution to  $(\star)$  is the eigenvector of the second smallest eigenvalue of  $\tilde{\mathbf{L}}$ .

#### *Algorithm for normalized cut*

1. Construct  $\mathbf{W} \leftarrow [w_{ij}]_{i,j=1}^m$ .
2. Compute  $\mathbf{D} \leftarrow \text{diag}(\mathbf{W}\mathbf{1})$ .
3. Compute  $\tilde{\mathbf{L}} \leftarrow \mathbf{I} - \mathbf{D}^{-\frac{1}{2}} \mathbf{W} \mathbf{D}^{-\frac{1}{2}}$ .
4. Compute the eigenvector  $\mathbf{x}_{m-1}$  of  $\tilde{\mathbf{L}}$  with the second smallest eigenvalue.
5. Compute  $\mathbf{z} \leftarrow \mathbf{D}^{-\frac{1}{2}} \mathbf{x}_{m-1}$ .
6. Set  $S \leftarrow \{\mathbf{x}_i : z_i > 0\}$ ,  $\bar{S} \leftarrow \{\mathbf{x}_i : z_i < 0\}$ . We get two clusters  $(S, \bar{S})$ .

If we want  $n > 2$  groups, we can perform normalized cut recursively:

Given  $k$  clusters

$$S_1 \sqcup S_2 \sqcup \dots \sqcup S_k = V$$

For each  $i = 1, \dots, k$ , we perform normalized cut on  $S_i$  and record the corresponding  $\lambda_{m-1}^{(i)}$ . We choose  $p = \arg_i \min_{1 \leq i \leq k} \lambda_{m-1}^{(i)}$ , cut  $S_p$  into  $S_p^{(1)} \sqcup S_p^{(2)}$  and get  $k + 1$  clusters

$$S_1 \sqcup S_2 \sqcup \dots \sqcup S_p^{(1)} \sqcup S_p^{(2)} \sqcup \dots \sqcup S_k = V$$

Repeat the steps above until we get  $n$  clusters.

## 6 Singular Value Decomposition

*Definition:* For any matrix  $\mathbf{A} \in \mathbb{R}^{m \times n}$ , the **singular value decomposition (SVD)** of  $\mathbf{A}$  is a factorization

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$$

where  $\mathbf{U} = [\mathbf{u}_1 \ \mathbf{u}_2 \ \dots \ \mathbf{u}_m] \in \mathbb{R}^{m \times m}$ ,  $\mathbf{V} = [\mathbf{v}_1 \ \mathbf{v}_2 \ \dots \ \mathbf{v}_n] \in \mathbb{R}^{n \times n}$  are orthogonal, and  $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$  is diagonal with diagonal entries  $\sigma_1 \geq \dots \geq \sigma_p \geq 0$ ,  $p = \min\{m, n\}$ .  $(\sigma_i)_{i=1}^p$  are called the **singular values** of  $\mathbf{A}$ , and  $(\mathbf{u}_i)_{i=1}^m, (\mathbf{v}_i)_{i=1}^n$  are called the **singular vectors**.

We also have the **compact singular value decomposition**

$$\mathbf{A} = [\mathbf{u}_1 \ \dots \ \mathbf{u}_r] \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_r \end{bmatrix} \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_r^\top \end{bmatrix}$$

where  $r = \text{rank}(\mathbf{A})$ .

The singular values and vectors of a matrix can be very useful in many aspects.

*Theorem:*

$$\text{Ran}(\mathbf{A}) = \text{span} \{ \mathbf{u}_1, \dots, \mathbf{u}_r \}$$

$$\text{Ran}(\mathbf{A}^\top) = \text{span} \{ \mathbf{v}_1, \dots, \mathbf{v}_r \}$$

$$\text{Ker}(\mathbf{A}) = \text{span} \{ \mathbf{v}_{r+1}, \dots, \mathbf{v}_n \}$$

$$\text{Ker}(\mathbf{A}^\top) = \text{span} \{ \mathbf{u}_{r+1}, \dots, \mathbf{u}_m \}$$

*Theorem:*

$$\|\mathbf{A}\|_F = \sqrt{\sigma_1^2 + \dots + \sigma_p^2}$$

$$\|\mathbf{A}\| = \sigma_1$$

*Theorem (Eckart-Young-Mirsky):* Given  $\mathbf{A} \in \mathbb{R}^{m \times n}$  with compact SVD  $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ . Define

$$\mathbf{A}_k = [\mathbf{u}_1 \ \dots \ \mathbf{u}_k] \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_k \end{bmatrix} \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_k^\top \end{bmatrix} = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

Then  $\mathbf{A}_k$  is the **best rank- $k$  approximation** of  $\mathbf{A}$  under  $\|\cdot\|$  or  $\|\cdot\|_F$ , i.e.,

$$\min_{\text{rank}(\mathbf{M}) \leq k} \|\mathbf{A} - \mathbf{M}\|_F = \|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sigma_{k+1}^2 + \dots + \sigma_n^2}$$

and

$$\min_{\text{rank}(\mathbf{M}) \leq k} \|\mathbf{A} - \mathbf{M}\| = \|\mathbf{A} - \mathbf{A}_k\| = \sigma_{k+1}$$

Now let's introduce some algorithms for singular value decomposition. Note that

$$\mathbf{A}^\top \mathbf{A} = \mathbf{V} \mathbf{\Sigma}^2 \mathbf{V}^\top \quad \text{and} \quad \mathbf{A} \mathbf{A}^\top = \mathbf{U} \mathbf{\Sigma}^2 \mathbf{U}^\top$$

This property enables us to convert the problem to eigenvalue problems of  $\mathbf{A}^\top \mathbf{A}$  and  $\mathbf{A} \mathbf{A}^\top$ .

*Task:* Given a matrix  $\mathbf{A}$ , find its largest singular value and corresponding singular vectors.

**Algorithm for SVD using power method**

1. Choose  $\mathbf{v}^{(0)}$  such that  $\|\mathbf{v}^{(0)}\| = 1$
2. For  $k = 1, 2, \dots$  until convergence

$$\mathbf{y}^{(k)} \leftarrow \mathbf{A}\mathbf{v}^{(k-1)}$$

$$\mathbf{u}^{(k)} \leftarrow \frac{\mathbf{y}^{(k)}}{\|\mathbf{y}^{(k)}\|}$$

$$\mathbf{z}^{(k)} \leftarrow \mathbf{A}^\top \mathbf{u}^{(k)}$$

$$\mathbf{v}^{(k)} \leftarrow \frac{\mathbf{z}^{(k)}}{\|\mathbf{z}^{(k)}\|}$$

$$\sigma^{(k)} \leftarrow \|\mathbf{A}\mathbf{v}^{(k)}\|$$

We get  $(\sigma^{(k)}, \mathbf{u}^{(k)}, \mathbf{v}^{(k)})$

*Remark:* Power iteration may fail to converge if  $\sigma_1 = \sigma_2$  or  $\mathbf{v}_1^\top \mathbf{v}^{(0)} = 0$ .

To get the singular value decomposition of  $\mathbf{A}$ , we can apply the QR algorithm.

We first use Householder transformation to reduce  $\mathbf{A}$  to upper bidiagonal (i.e.  $\begin{bmatrix} * & * & & \\ & * & & \\ & & \ddots & \\ & & & * \\ & & & & * \end{bmatrix}$ ),

denoted by  $\mathbf{B}$ . One can easily show that  $\mathbf{B}^\top \mathbf{B}$  and  $\mathbf{B}\mathbf{B}^\top$  are tridiagonal. By applying QR algorithm discussed in the previous section, we can obtain the eigenvalues and eigenvectors of  $\mathbf{A}^\top \mathbf{A}$  and  $\mathbf{A}\mathbf{A}^\top$ , and hence we get  $\mathbf{U}$ ,  $\mathbf{V}$  and  $\mathbf{\Sigma}$ .

Now let's see some examples. First we revisit the least square problems.

*Task:* Given a matrix  $\mathbf{A} \in \mathbb{R}^{m \times n}$  with  $m > n$  and a vector  $\mathbf{b} \in \mathbb{R}^n$ . We want to solve

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2$$

*Theorem:* Given a matrix  $\mathbf{A} \in \mathbb{R}^{m \times n}$  with  $m > n$  and a vector  $\mathbf{b} \in \mathbb{R}^n$ , the solution to the least square problem  $\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|^2$  is given by

$$\hat{\mathbf{x}} = \mathbf{V}\mathbf{\Sigma}^+ \mathbf{U}^\top \mathbf{b}$$

Where  $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$  is its compact SVD, and  $\mathbf{\Sigma}^+ \in \mathbb{R}^{n \times m}$  is a diagonal matrix with

$$(\mathbf{\Sigma}^+)_{ii} = \begin{cases} \frac{1}{\sigma_i} & \sigma_i > 0 \\ 0 & \sigma_i = 0 \end{cases}$$

The matrix  $\mathbf{V}\mathbf{\Sigma}^+ \mathbf{U}^\top$  is called the *pseudo-inverse* of  $\mathbf{A}$ , often denoted by  $\mathbf{A}^+$ .

Next we introduce the famous *principal component analysis (PCA)*.

*Task:* Let  $\mathbf{A} = [\mathbf{a}_1 \ \mathbf{a}_2 \ \cdots \ \mathbf{a}_n] \in \mathbb{R}^{m \times n}$  be a dataset matrix, where each column represents a data point. We want to find the common features  $\mathbf{x}_1, \dots, \mathbf{x}_k \in \mathbb{R}^m$  of these data points, with  $k \ll m, n$ .

One would expect that for any  $i = 1, \dots, n$ ,  $\mathbf{a}_i$  is approximately a linear combination of  $\mathbf{x}_1, \dots, \mathbf{x}_k$ , i.e.,

$$\mathbf{a}_i \approx \sum_{j=1}^k c_{ij} \mathbf{x}_j \quad \forall i$$

Define  $\mathbf{X} = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_k] \in \mathbb{R}^{m \times k}$  and  $\mathbf{C} = [c_{ij}]_{i,j=1}^{k,n} \in \mathbb{R}^{k \times n}$ . We should have  $\mathbf{A} \approx \mathbf{XC}$ .

In other word, we want to solve

$$\min_{\substack{\mathbf{X} \in \mathbb{R}^{m \times k} \\ \mathbf{C} \in \mathbb{R}^{k \times n}}} \|\mathbf{A} - \mathbf{XC}\|_F^2$$

Since  $\text{rank}(\mathbf{XC}) = \min\{m, k, n\} = k$ , the optimization problem above is equivalent to

$$\min_{\text{rank}(\mathbf{A}') \leq k} \|\mathbf{A} - \mathbf{A}'\|_F^2$$

which is exactly the best low-rank approximation, whose solution is given by  $\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ , where  $(\sigma_i, \mathbf{u}_i, \mathbf{v}_i)_{i=1}^k$  are the top  $k$  singular triplets of  $\mathbf{A}$ . To further decompose  $\mathbf{A}_k$  into  $\mathbf{XC}$ , we can just set

$$\mathbf{X} = [\mathbf{u}_1 \ \mathbf{u}_2 \ \cdots \ \mathbf{u}_k] \quad \text{and} \quad \mathbf{C} = \begin{bmatrix} \sigma_1 \mathbf{v}_1^\top \\ \vdots \\ \sigma_k \mathbf{v}_k^\top \end{bmatrix}$$

The full algorithm goes as follows:

#### *Algorithm for PCA*

1. Compute the top  $k$  singular triplets  $(\sigma_i, \mathbf{u}_i, \mathbf{v}_i)_{i=1}^k$  of  $\mathbf{A}$ .
2. Set  $\mathbf{X} \leftarrow [\mathbf{u}_1 \ \mathbf{u}_2 \ \cdots \ \mathbf{u}_k]$  and  $\mathbf{C} \leftarrow \begin{bmatrix} \sigma_1 \mathbf{v}_1^\top \\ \vdots \\ \sigma_k \mathbf{v}_k^\top \end{bmatrix}$ .
3. For any new data point  $\mathbf{a}_{\text{new}}$ , we get  $\mathbf{c}_{\text{new}} = \begin{bmatrix} \mathbf{u}_1^\top \mathbf{a}_{\text{new}} \\ \vdots \\ \mathbf{u}_k^\top \mathbf{a}_{\text{new}} \end{bmatrix}$ .

Our last example is about *matrix completion*.

Assume we have  $m$  people and  $n$  movies. Each person gives a rating on each of the movies. We construct the rating matrix  $\mathbf{M} = [m_{ij}]_{i,j=1}^{m,n} \in \mathbb{R}^{m \times n}$ , where  $m_{ij}$  represents the rating of user  $i$  to movie  $j$ . However, only a small proportion of  $\mathbf{M}$  is known. We want to complete the matrix  $\mathbf{M}$ .

*Task:* We want to find a complete  $\mathbf{M}$  from  $\mathbf{M}|_{\Omega} = \{m_{ij} : (i, j) \in \Omega\}$ , where  $\Omega \subset \{(i, j)\}_{i,j=1}^{m,n}$  with  $|\Omega| \ll mn$ .

We build our model under the following assumption:

The ratings of each movie are only related to  $r$  factors, i.e.,

$$m_{ij} = \sum_{k=1}^r p_{ik} q_{jk}$$

where  $p_{ik}$  represents the preference of user  $i$  to factor  $k$ , and  $q_{jk}$  represents the performance of movie  $j$  on factor  $k$ .

Let  $\mathbf{P} = [p_{ik}]_{i,k=1}^{m,r}$ ,  $\mathbf{Q} = [q_{jk}]_{j,k=1}^{n,r}$ , then  $\mathbf{M} = \mathbf{P}\mathbf{Q}^{\top}$ . We can use an iterative approach.

#### **Algorithm for matrix completion**

1. Choose  $\mathbf{M}_0 \in \mathbb{R}^{m \times n}$  such that  $(\mathbf{M}_0)_{ij} = m_{ij} \forall (i, j) \in \Omega$ .

2. For  $k = 0, 1, \dots$

Perform SVD on  $\mathbf{M}_k$  and get  $\mathbf{M}_k = \sum_{i=1}^{\min\{m,n\}} \sigma_i \mathbf{u}_i \mathbf{v}_i^{\top}$ .

$\mathbf{R}_k \leftarrow \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^{\top}$  ( $\mathbf{R}_k$  is best rank- $r$  approximation of  $\mathbf{M}_k$ )

For  $i = 1, \dots, m, j = 1, \dots, n$

$$(\mathbf{M}_{k+1})_{ij} \leftarrow \begin{cases} m_{ij} & (i, j) \in \Omega \\ (\mathbf{R}_k)_{ij} & (i, j) \notin \Omega \end{cases}$$

And we get the completed matrix  $\mathbf{M}_k$ .